DriveSpark speaking guide

> Current code, 10 November 2026. Four standard OOP pillars plus exception handling as a course topic. Confirm lecturer terminology. NEEDS CHECK marks unconfirmed history or behavior.

Final clean verify: 694 tests, zero failures, errors or skips. Use disposable data for demonstrations.

Project-level questions

What does our system do?

In our system, I help a school manage students, lesson packages, bookings, vehicles and reviews. I give admins reports and saved website enquiries to read.

Show in code: HomeController.java - `showHomePage`; AdminReportService.java - `business`; ContactService.java - `messagesForAdmin`

Show in browser: Home > Packages > View package

What are the four roles?

I explain visitors, students, instructors and admins as four user experiences. I store only three Role enum values because a visitor is someone who is not logged in.

Show in code: Role.java - `class declaration`; NavigationInterceptor.java - `postHandle`

Why Spring Boot?

I use Spring Boot to start the Java web application and connect its components. I use constructor injection so Spring supplies the objects each controller needs.

Show in code: DrivesparkAcademyApplication.java - `main`; BookingController.java - `BookingController`

How does MVC work here?

I let controllers accept requests, services check rules and repositories read or save data. I pass a model to a JSP view to display the result.

Show in code: BookingController.java - `list`; BookingService.java - `getOwnBookings`; BookingRepository.java - `readAllStrict`

Show in browser: Student > My lessons

Why file-based storage?

I store one JSON object per line so the records are easy to inspect and run locally. I accept that this has fewer safety and querying features than a database.

Show in code: FileRepository.java - `readRecords`; FileRepository.java - `writeRecords`

Are file writes transactions?

I do not have a transaction across several files. I use file replacement and locks inside one JVM, but a crash between two file updates can still leave partial work.

Show in code: FileRepository.java - `writeRecords`; AccountWriteCoordinator.java - `execute`; EnrollmentService.java - `markPaid`

How are passwords stored?

I store a salted BCrypt hash instead of the original password. I use matches to check a login and never need to decrypt a password.

Show in code: PasswordHasher.java - `hash`; PasswordHasher.java - `matches`

How do sessions work?

I keep the logged-in user ID in a server-side HTTP session. I rotate the session on login and reload the account so a deleted or suspended account does not keep normal access.

Show in code: AuthSessionService.java - `loginWithResult`; AuthSessionService.java - `getCurrentUser`

Show in browser: Log in > Account > Log out

What is CSRF protection?

I put a session token in forms that change data. I reject unsafe requests with missing or wrong tokens so another website cannot simply submit a form as the signed-in user.

Show in code: CsrfTokenService.java - `isValid`; CsrfFilter.java - `doFilterInternal`

How do we protect admin pages?

I use access annotations and an interceptor to check login and role. I also check role and ownership in services before exposing or changing a record.

Show in code: AccessInterceptor.java - `preHandle`; BookingService.java - `getBookingDetails`

Where is validation done?

I use browser rules to help the user enter details. I repeat important checks in Java because someone can bypass the browser.

Show in code: Student.java - `setPhone`; LearningPackage.java - `setPrice`; BookingService.java - `requestLesson`

How do error pages help?

I convert known exceptions into safe pages with useful status codes and recovery links. I keep private record contents and internal paths out of the user message.

Show in code: GlobalExceptionHandler.java - `handleValidation`; GlobalExceptionHandler.java - `handleStorage`; ErrorPageSupport.java - `prepare`

How does payment confirmation work?

I first save an enrollment awaiting payment without granting lessons. I let an admin confirm received payment, then start validity and grant the package lesson count.

Show in code: EnrollmentService.java - `enroll`; EnrollmentService.java - `markPaid`; EnrollmentService.java - `activateLessons`

Show in browser: Student 8 > My enrollments > Payment instructions; admin > Awaiting payment

Why can students register freely?

I create a new student as Active so they can log in and choose a package. I still require a paid usable enrollment before booking, and admins can manage account status.

Show in code: AuthService.java - `registerStudent`; PaidEnrollmentGuard.java - `requirePaid`

Show in browser: Register > Log in > Packages

How is data seeded?

I enable the loader only with the demo setting and a fresh empty folder. I add missing records in dependency order, preserve handled requests and spent balances, and calculate ratings from approved reviews.

Show in code: DemoDataSeedRunner.java - `run`; DemoDataSeedRunner.java - `syncInstructorRatings`; DemoDataSeedRunnerTest.java - `seedCanRunTwiceWithoutDuplicateRecords`

How was the Tailwind UI built?

I use Tailwind utilities and shared CSS classes, then build a local stylesheet with npm run build:css. I use white account cards on soft grey, dark navigation and amber actions, while public photos stay cinematic.

Show in code: package.json - `build:css`; app.css - `body.ds-account-page`

What do reports count?

I count saved records and include only Paid enrollment amounts as revenue. I group revenue by enrollment month because I do not store a separate payment timestamp.

Show in code: AdminReportService.java - `business`; AdminReportService.java - `money`

Show in browser: Admin > Reports

How many tests do we have?

I can show the recorded final clean verify result of 694 tests with zero failures, errors or skips. I separate automated tests from browser checks and their stated limits.

Show in code: pom.xml - `build dependency`; AdminReportServiceTest.java - `businessCsvUsesNamesHeadersTotalsAndBom`

What are the custom ordering algorithms?

I use hand-written bubble sort to order instructors by a chosen detail. I use a linked FIFO queue for pending unassigned requests and rebuild its order from saved creation times.

Show in code: InstructorSorter.java - `sort`; BookingRequestQueue.java - `BookingRequestQueue`

What would I improve?

I would add a relational database with transactions and stronger audit records. I would then add real payment and email integrations with proper failure handling.

Show in code: FileRepository.java - `writeRecords`; PasswordController.java - `requestReset`

IT26101135: Accounts, students and integration

What is my module responsible for?

In our system, I use separate classes for the accounts, students and integration module. I keep web requests, rules and storage in separate classes.

Show in code: StudentProfileController.java - `StudentProfileController`; StudentProfileService.java - `StudentProfileService`

Where are Create and Read?

I create through AuthService using registerStudent and read through StudentProfileService using getOwnProfile. I let repositories handle files after service checks.

Show in code: AuthService.java - `registerStudent`; StudentProfileService.java - `getOwnProfile`

Where are Update and Delete?

I update through StudentProfileService using updateOwnProfile and delete through StudentAccountService using deleteOwnAccount. I apply deletion rules instead of breaking saved lesson or payment history.

Show in code: StudentProfileService.java - `updateOwnProfile`; StudentAccountService.java - `deleteOwnAccount`

Where did I use encapsulation?

I keep Student fields private and use checked methods to control changes. I use setPhone to reject invalid values instead of trusting a form.

Show in code: Student.java - `setPhone`

Where did I use abstraction?

I use abstract User for common account details. I require each account child to supply its own role and dashboard path.

Show in code: User.java - `getRole`; User.java - `getDashboardPath`

Where did I use inheritance?

I make Student extend User so it reuses account name, email and password-hash rules. I use Admin and InstructorAccount as other User children.

Show in code: Student.java - `class declaration`; Admin.java - `class declaration`; InstructorAccount.java - `class declaration`

Where did I use polymorphism?

I call getDashboardPath through a User reference after login. I get the student, instructor or admin destination from the actual account class.

Show in code: AuthController.java - `login`; User.java - `getDashboardPath`

How did I use exception handling as the fifth course topic?

I use DuplicateRecordException when this module cannot accept a requested change. I let the shared handler explain it safely instead of exposing file details.

Show in code: StudentProfileService.java - `ensureAvailableEmail`; GlobalExceptionHandler.java - `handleDuplicate`

How do I explain Single Responsibility and Open/Closed?

I keep profile HTTP handling in StudentProfileController and rules in StudentProfileService. I can add a User child without rewriting existing children, but a new role still needs enum and access changes.

Show in code: StudentProfileController.java - `updateProfile`; User.java - `class declaration`

How do I explain Liskov, Interface Segregation and Dependency Inversion?

I can use Student as User while keeping a valid role and destination. I give profile pages the smaller StudentProfiles contract. I inject that interface into StudentProfileController rather than constructing its service there.

Show in code: Student.java - `getRole`; StudentProfiles.java - `getOwnProfile`; StudentProfileController.java - `StudentProfileController`

How do I explain Registration, login and sessions?

I register a student as Active with a unique email and hashed password. I create a session only after the password and account status pass. I keep payment eligibility separate from account registration.

Show in code: AuthService.java - `registerStudent`; AuthSessionService.java - `loginWithResult`

What should I demonstrate honestly?

I can demonstrate the accounts, students and integration pages using disposable seeded data. I use server-side rules and tests as evidence, and mark an unconfirmed demonstration NEEDS CHECK.

Show in code: StudentProfileService.java - `StudentProfileService`

Show in browser: Open Register from the login page > Return using Log in > Log in as student1 > Open Account settings > Open Change password without submitting

IT26101133: Instructors, availability and bubble sort

What is my module responsible for?

In our system, I use separate classes for the instructors, availability and bubble sort module. I keep web requests, rules and storage in separate classes.

Show in code: InstructorAvailabilityController.java - `InstructorAvailabilityController`; InstructorService.java - `InstructorService`

Where are Create and Read?

I create through InstructorAccountService using create and read through InstructorService using sortInstructors. I let repositories handle files after service checks.

Show in code: InstructorAccountService.java - `create`; InstructorService.java - `sortInstructors`

Where are Update and Delete?

I update through InstructorAccountService using update and delete through InstructorAccountService using delete. I apply deletion rules instead of breaking saved lesson or payment history.

Show in code: InstructorAccountService.java - `update`; InstructorAccountService.java - `delete`

Where did I use encapsulation?

I keep Instructor fields private and use checked methods to control changes. I use setYearsOfExperience to reject invalid values instead of trusting a form.

Show in code: Instructor.java - `setYearsOfExperience`

Where did I use abstraction?

I use InstructorOrder to describe how two instructors are compared. I let the sorter use that contract rather than put every comparison inside the swap loop.

Show in code: InstructorOrder.java - `compare`; InstructorSorter.java - `swapNeighbours`

Where did I use inheritance?

I make InstructorRepository extend FileRepository for shared file operations. I keep Instructor as a separate profile because it does not extend the account User class.

Show in code: InstructorRepository.java - `class declaration`; Instructor.java - `class declaration`

Where did I use polymorphism?

I call the same compare method with name, experience or rating comparison objects. I get different decisions while keeping the same bubble-sort passes.

Show in code: InstructorSorter.java - `swapNeighbours`; InstructorNameOrder.java - `compare`; InstructorExperienceOrder.java - `compare`; InstructorRatingOrder.java - `compare`

How did I use exception handling as the fifth course topic?

I use DuplicateRecordException when this module cannot accept a requested change. I let the shared handler explain it safely instead of exposing file details.

Show in code: InstructorScheduleService.java - `ensureNoOverlap`; GlobalExceptionHandler.java - `handleDuplicate`

How do I explain Single Responsibility and Open/Closed?

I give InstructorSorter only the sorting job. I can add another InstructorOrder without rewriting the swap passes, although chooseOrder must still select the new option.

Show in code: InstructorSorter.java - `sort`; InstructorSorter.java - `chooseOrder`

How do I explain Liskov, Interface Segregation and Dependency Inversion?

I let each InstructorOrder return a negative, zero or positive comparison result. I keep that interface to one compare method. I inject InstructorScheduleOperations into the availability controller.

Show in code: InstructorNameOrder.java - `compare`; InstructorOrder.java - `compare`; InstructorAvailabilityController.java - `InstructorAvailabilityController`

How do I explain Bubble sort, steps and complexity?

I compare neighbours and swap them when they are in the wrong order. For ascending [8, 3, 12], I get [3, 8, 12] after the first pass and stop after a pass without swaps. I have O(n squared) worst-case time and O(n) best-case time, plus O(n) space for the copied list.

Show in code: InstructorSorter.java - `sort`; InstructorSorter.java - `swapNeighbours`

What should I demonstrate honestly?

I can demonstrate the instructors, availability and bubble sort pages using disposable seeded data. I use server-side rules and tests as evidence, and mark an unconfirmed demonstration NEEDS CHECK.

Show in code: InstructorService.java - `InstructorService`

Show in browser: Open Instructors > Select Experience > Choose descending order > Click Sort > Open Packages

IT26101134: Bookings, FIFO and conflict rules

What is my module responsible for?

In our system, I use separate classes for the bookings, fifo and conflict rules module. I keep web requests, rules and storage in separate classes.

Show in code: BookingController.java - `BookingController`; BookingService.java - `BookingService`

Where are Create and Read?

I create through BookingService using requestLesson and read through BookingService using getOwnBookings. I let repositories handle files after service checks.

Show in code: BookingService.java - `requestLesson`; BookingService.java - `getOwnBookings`

Where are Update and Delete?

I update through BookingService using rescheduleOwnBooking and delete through BookingService using deleteCancelledBooking. I apply deletion rules instead of breaking saved lesson or payment history.

Show in code: BookingService.java - `rescheduleOwnBooking`; BookingService.java - `deleteCancelledBooking`

Where did I use encapsulation?

I keep Booking fields private and use checked methods to control changes. I use Booking to reject invalid values instead of trusting a form.

Show in code: Booking.java - `Booking`

Where did I use abstraction?

I separate StudentBookingOperations from AdminBookingOperations. I describe list ordering through BookingOrder without exposing the queue nodes.

Show in code: StudentBookingOperations.java - `class declaration`; AdminBookingOperations.java - `class declaration`; BookingOrder.java - `order`

Where did I use inheritance?

I make BookingRepository extend FileRepository for shared storage. I use a private linked Node inside the queue rather than claiming the queue inherits from Booking.

Show in code: BookingRepository.java - `class declaration`; BookingRequestQueue.java - `class declaration`

Where did I use polymorphism?

I call order through BookingOrder references for pending-request and lesson-time views. I remember that the pending view also filters requests, so those views do not have identical business meanings.

Show in code: BookingService.java - `getAllBookings`; PendingRequestOrder.java - `order`; LessonTimeOrder.java - `order`

How did I use exception handling as the fifth course topic?

I use ValidationException when this module cannot accept a requested change. I let the shared handler explain it safely instead of exposing file details.

Show in code: BookingService.java - `parseFilterDate`; GlobalExceptionHandler.java - `handleValidation`

How do I explain Single Responsibility and Open/Closed?

I keep conflict checks in BookingConflictChecker and waiting order in BookingRequestQueue. I can add another BookingOrder without rewriting the existing ordering classes, but the caller must choose the right view.

Show in code: BookingConflictChecker.java - `conflictReason`; BookingOrder.java - `order`

How do I explain Liskov, Interface Segregation and Dependency Inversion?

I preserve the shared save contract in BookingRepository. I separate student and admin booking contracts. I inject StudentBookingOperations into BookingController.

Show in code: BookingRepository.java - `save`; StudentBookingOperations.java - `class declaration`; BookingController.java - `BookingController`

How do I explain FIFO, enqueue, dequeue and complexity?

I enqueue at the tail and dequeue the oldest head, each in O(1) time. I use FIFO so later unassigned requests do not jump ahead. I rebuild the queue with stable insertion sorting, which can take O(n squared), and position lookup takes O(n).

Show in code: BookingRequestQueue.java - `enqueue`; BookingRequestQueue.java - `dequeue`; BookingRequestQueue.java - `positionOf`; BookingService.java - `processNext`

What should I demonstrate honestly?

I can demonstrate the bookings, fifo and conflict rules pages using disposable seeded data. I use server-side rules and tests as evidence, and mark an unconfirmed demonstration NEEDS CHECK.

Show in code: BookingService.java - `BookingService`

Show in browser: Open My lessons as student1 > Open the first booking details > Inspect its waiting-list position > Click Back to bookings > Open History

IT26101132: Vehicles and seed data

What is my module responsible for?

In our system, I use separate classes for the vehicles and seed data module. I keep web requests, rules and storage in separate classes.

Show in code: VehicleController.java - `VehicleController`; VehicleService.java - `VehicleService`

Where are Create and Read?

I create through VehicleService using createVehicle and read through VehicleService using getAllVehicles. I let repositories handle files after service checks.

Show in code: VehicleService.java - `createVehicle`; VehicleService.java - `getAllVehicles`

Where are Update and Delete?

I update through VehicleService using updateVehicle and delete through VehicleService using deleteVehicle. I apply deletion rules instead of breaking saved lesson or payment history.

Show in code: VehicleService.java - `updateVehicle`; VehicleService.java - `deleteVehicle`

Where did I use encapsulation?

I keep Vehicle fields private and use checked methods to control changes. I use setNextServiceDate to reject invalid values instead of trusting a form.

Show in code: Vehicle.java - `setNextServiceDate`

Where did I use abstraction?

I expose fleet actions through VehicleOperations. I use VehicleStore for storage tasks instead of putting JSON file code in the controller.

Show in code: VehicleOperations.java - `class declaration`; VehicleStore.java - `class declaration`; VehicleController.java - `VehicleController`

Where did I use inheritance?

I make VehicleRepository extend FileRepository to reuse storage, validation and ID counters. I do not copy the JSON writer into Vehicle.

Show in code: VehicleRepository.java - `class declaration`

Where did I use polymorphism?

I use a VehicleStore reference in VehicleService with VehicleRepository as the actual object. I call its storage methods while keeping the parent file rules.

Show in code: VehicleService.java - `VehicleService`; VehicleRepository.java - `save`

How did I use exception handling as the fifth course topic?

I use DuplicateRecordException when this module cannot accept a requested change. I let the shared handler explain it safely instead of exposing file details.

Show in code: VehicleService.java - `ensureRegistrationNumberIsUnique`; GlobalExceptionHandler.java - `handleDuplicate`

How do I explain Single Responsibility and Open/Closed?

I keep vehicle rules in VehicleService and file writing in VehicleRepository. I can add another typed repository without rewriting the shared FileRepository writer.

Show in code: VehicleService.java - `deleteVehicle`; VehicleRepository.java - `class declaration`; FileRepository.java - `writeRecords`

How do I explain Liskov, Interface Segregation and Dependency Inversion?

I delegate VehicleRepository updates to the same parent checks. I give the controller the VehicleOperations fleet contract. I inject that interface instead of constructing VehicleService in the controller.

Show in code: VehicleRepository.java - `update`; VehicleOperations.java - `class declaration`; VehicleController.java - `VehicleController`

How do I explain Expiry warnings and safe seeding?

I show separate expired or overdue labels and warnings for dates within thirty days. I seed missing records without refilling spent lessons. I use fictional phone and NIC patterns accepted by the app, not real allocated identities.

Show in code: Vehicle.java - `isInsuranceWarning`; Vehicle.java - `isServiceWarning`; DemoDataSeedRunner.java - `run`

What should I demonstrate honestly?

I can demonstrate the vehicles and seed data pages using disposable seeded data. I use server-side rules and tests as evidence, and mark an unconfirmed demonstration NEEDS CHECK.

Show in code: VehicleService.java - `VehicleService`

Show in browser: Open Vehicles as admin > Open Edit vehicle for the Alto > Return through Vehicles in the sidebar > Open Edit vehicle for the Honda Dio > Return to Vehicles and inspect warnings

IT26101136: Packages, enrollment and payment

What is my module responsible for?

In our system, I use separate classes for the packages, enrollment and payment module. I keep web requests, rules and storage in separate classes.

Show in code: LearningPackageController.java - `LearningPackageController`; LearningPackageService.java - `LearningPackageService`

Where are Create and Read?

I create through LearningPackageService using createPackage and read through LearningPackageService using listPackages. I let repositories handle files after service checks.

Show in code: LearningPackageService.java - `createPackage`; LearningPackageService.java - `listPackages`

Where are Update and Delete?

I update through LearningPackageService using updatePackage and delete through LearningPackageService using deletePackage. I apply deletion rules instead of breaking saved lesson or payment history.

Show in code: LearningPackageService.java - `updatePackage`; LearningPackageService.java - `deletePackage`

Where did I use encapsulation?

I keep LearningPackage fields private and use checked methods to control changes. I use setPrice to reject invalid values instead of trusting a form.

Show in code: LearningPackage.java - `setPrice`

Where did I use abstraction?

I use LearningPackageCatalog for package-reading tasks and EnrollmentOperations for student enrollment tasks. I keep admin payment tasks in EnrollmentAdministration.

Show in code: LearningPackageCatalog.java - `class declaration`; EnrollmentOperations.java - `class declaration`; EnrollmentAdministration.java - `class declaration`

Where did I use inheritance?

I make LearningPackageRepository extend FileRepository for storage rules. I do not make Enrollment extend LearningPackage because they are different records.

Show in code: LearningPackageRepository.java - `class declaration`; Enrollment.java - `class declaration`

Where did I use polymorphism?

I let LearningPackage and Enrollment implement the same LearningRecord display contract. I can call their display methods through that contract, although current pages use presentation helpers for resolved package names.

Show in code: LearningRecord.java - `getDisplayName`; LearningPackage.java - `getDisplayName`; Enrollment.java - `getDisplayName`

How did I use exception handling as the fifth course topic?

I use DuplicateRecordException when this module cannot accept a requested change. I let the shared handler explain it safely instead of exposing file details.

Show in code: LearningPackageService.java - `ensureUniqueName`; GlobalExceptionHandler.java - `handleDuplicate`

How do I explain Single Responsibility and Open/Closed?

I build read-only labels in EnrollmentPresentationService while EnrollmentService handles payment rules. I can add another LearningRecord without rewriting existing records, although a page must choose to use it.

Show in code: EnrollmentPresentationService.java - `addStudent`; LearningRecord.java - `getDisplayName`

How do I explain Liskov, Interface Segregation and Dependency Inversion?

I preserve the shared storage contract in LearningPackageRepository. I separate student EnrollmentOperations from admin EnrollmentAdministration. I inject EnrollmentOperations into EnrollmentController.

Show in code: LearningPackageRepository.java - `save`; EnrollmentAdministration.java - `class declaration`; EnrollmentController.java - `EnrollmentController`

How do I explain Payment confirmation and packages?

I recheck the waiting request, student and course inside the shared account lock. I activate lessons and save the Paid receipt, with validity starting on confirmation day. I do not claim a transaction because those are separate file updates.

Show in code: EnrollmentService.java - `markPaid`; EnrollmentService.java - `activateLessons`; AccountWriteCoordinator.java - `execute`

What should I demonstrate honestly?

I can demonstrate the packages, enrollment and payment pages using disposable seeded data. I use server-side rules and tests as evidence, and mark an unconfirmed demonstration NEEDS CHECK.

Show in code: LearningPackageService.java - `LearningPackageService`

Show in browser: Open My enrollments as student8 > Open Payment instructions > Return to My enrollments > Open Packages > Open View package

IT26101137: Reviews, ratings and reports

What is my module responsible for?

In our system, I use separate classes for the reviews, ratings and reports module. I keep web requests, rules and storage in separate classes.

Show in code: ReviewController.java - `ReviewController`; ReviewService.java - `ReviewService`

Where are Create and Read?

I create through ReviewService using createReview and read through ReviewService using getVisibleReviews. I let repositories handle files after service checks.

Show in code: ReviewService.java - `createReview`; ReviewService.java - `getVisibleReviews`

Where are Update and Delete?

I update through ReviewService using updateOwnReview and delete through ReviewService using deleteOwnReview. I apply deletion rules instead of breaking saved lesson or payment history.

Show in code: ReviewService.java - `updateOwnReview`; ReviewService.java - `deleteOwnReview`

Where did I use encapsulation?

I keep Review fields private and use checked methods to control changes. I use Review to reject invalid values instead of trusting a form.

Show in code: Review.java - `Review`

Where did I use abstraction?

I expose student feedback through ReviewOperations and moderation through ReviewModeration. I expose report tables through ReportCatalog so the controller does not calculate them.

Show in code: ReviewOperations.java - `class declaration`; ReviewModeration.java - `class declaration`; ReportCatalog.java - `business`

Where did I use inheritance?

I make ReviewRepository extend FileRepository for shared file operations. I keep Review as a model implementing Identifiable rather than a child of a service.

Show in code: ReviewRepository.java - `class declaration`; Review.java - `class declaration`

Where did I use polymorphism?

I call business through the ReportCatalog reference in the report controller. I receive the real AdminReportService object supplied by Spring.

Show in code: AdminReportController.java - `show`; AdminReportService.java - `business`

How did I use exception handling as the fifth course topic?

I use DuplicateRecordException when this module cannot accept a requested change. I let the shared handler explain it safely instead of exposing file details.

Show in code: ReviewService.java - `ensureNoDuplicate`; GlobalExceptionHandler.java - `handleDuplicate`

How do I explain Single Responsibility and Open/Closed?

I calculate tables in AdminReportService and write downloads in AdminReportController. I can supply another ReportCatalog implementation without putting calculation rules in the controller.

Show in code: AdminReportService.java - `business`; AdminReportController.java - `export`; ReportCatalog.java - `business`

How do I explain Liskov, Interface Segregation and Dependency Inversion?

I preserve the FileRepository contract in ReviewRepository. I split ordinary review operations from ReviewModeration. I inject ReviewOperations and ReportCatalog into their controllers.

Show in code: ReviewRepository.java - `save`; ReviewModeration.java - `class declaration`; ReviewController.java - `ReviewController`; AdminReportController.java - `AdminReportController`

How do I explain Paid-only revenue and business CSV?

I add captured enrollment amounts only when isPaid is true and format them with two decimals. I write a UTF-8 BOM, a business header, named sections and totals. I quote cells, neutralize formula prefixes, and offer full and section downloads.

Show in code: AdminReportService.java - `business`; AdminReportController.java - `export`; AdminReportController.java - `csv`

What should I demonstrate honestly?

I can demonstrate the reviews, ratings and reports pages using disposable seeded data. I use server-side rules and tests as evidence, and mark an unconfirmed demonstration NEEDS CHECK.

Show in code: ReviewService.java - `ReviewService`

Show in browser: Open Reports as admin > Download a section CSV > Open Print report > Cancel printing after inspecting it > Download full CSV

Tricky what-if questions

What if two admins confirm the same payment?

I serialize the operation and reload the request inside the shared account lock. I grant lessons once; the second attempt sees it was already handled. I only have this protection in one JVM, not across app instances.

Show in code: EnrollmentService.java - `markPaid`; EnrollmentService.java - `requireAwaiting`; AccountWriteCoordinator.java - `execute`

What if a student double-clicks Book?

I disable an accepted submit in JavaScript and recheck conflicts and commitments under the shared booking lock. I reject a repeated unavailable overlapping time, but I do not have a general idempotency-key API.

Show in code: BookingService.java - `requestLesson`; BookingService.java - `requireCommitmentRoom`

What if a vehicle goes into maintenance after assignment?

I save the status but do not automatically cancel all existing bookings. I recheck conflicts when assigning or accepting, while completion is not a fresh vehicle-inspection check. I would ask the admin to reassign or cancel the affected lesson.

Show in code: VehicleService.java - `updateVehicle`; BookingService.java - `acceptAssignedLesson`; BookingService.java - `completeAssignedLesson`

What if a file is corrupted?

I use strict reads for important decisions, so damaged data raises StorageException instead of being silently trusted. I show a safe error page and would repair or restore a trusted copy before continuing.

Show in code: FileRepository.java - `readAllStrict`; GlobalExceptionHandler.java - `handleStorage`

What if an instructor adds leave on a confirmed lesson date?

I block leave over confirmed lessons or assigned pending lessons. I ask an administrator to resolve those lessons first. I use the shared app lock for schedule changes and booking writes.

Show in code: InstructorScheduleService.java - `validateLeave`; InstructorScheduleService.java - `deleteAvailability`; BookingService.java - `acceptAssignedLesson`

What if someone changes an ID in the URL?

I check role and record ownership instead of trusting the URL. I return a forbidden, missing-record or validation response when access is not allowed.

Show in code: BookingService.java - `getBookingDetails`; EnrollmentService.java - `requireOwnEnrollment`

What if the app restarts?

I keep records and counters in files and rebuild the queue from them. I lose ordinary in-memory sessions and reset codes. I do not use seeding to overwrite existing decisions.

Show in code: BookingRequestQueue.java - `BookingRequestQueue`; PasswordManagementService.java - `createResetToken`; DemoDataSeedRunner.java - `run`

What if a student cancels less than 24 hours before a lesson?

I reject the change with the default twenty-four-hour notice rule. I leave the booking intact; the admin has a separate cancellation operation for appropriate cases.

Show in code: BookingService.java - `cancelOwnBooking`; BookingService.java - `forceCancel`; BookingCutoffTest.java - `class declaration`

What if someone completes a future lesson?

I do not give students the completion operation. I block completion and no-show before lesson end with the default attendance setting. I show the lesson end time and disable attendance buttons until then. I allow an explicit override only for an isolated preview.

Show in code: BookingService.java - `completeAssignedLesson`; Booking.java - `complete`; AttendancePresentationAdvice.java - `isAvailable`

What if the package is unpaid, expired or used up?

I refuse booking when a paid receipt cannot supply usable lessons or cover the lesson date. I allow renewal of an expired or used-up package only with no open lesson and one payment request at a time. I keep old receipts and recheck eligibility when an administrator confirms payment.

Show in code: PaidEnrollmentGuard.java - `problemForLesson`; EnrollmentService.java - `renewalProblem`; EnrollmentService.java - `markPaid`

What if the course price changes later?

I keep the amount captured on an existing enrollment. I use that amount for revenue, so a catalog price change does not rewrite old paid amounts.

Show in code: Enrollment.java - `getAmount`; AdminReportService.java - `business`

What if a form has no CSRF token?

I reject the unsafe request before the controller changes data. I ask the user to reload the form rather than disabling the check.

Show in code: CsrfFilter.java - `doFilterInternal`; CsrfFilterTest.java - `class declaration`

What if someone deletes a referenced vehicle or package?

I refuse deletions that would break lesson or enrollment references. I use active or availability status where appropriate instead of deleting history.

Show in code: VehicleService.java - `deleteVehicle`; LearningPackageService.java - `deletePackage`

What if a review is edited or hidden?

I return an edited student review to pending approval. I calculate averages from approved reviews, so pending or hidden comments do not count.

Show in code: ReviewService.java - `updateOwnReview`; ReviewService.java - `recalculateRating`; AdminReportService.java - `business`

What if a crash occurs between balance and receipt writes?

I do not have rollback across those files, so partial state is possible. I would inspect the affected records rather than pretend the in-process lock is a transaction.

Show in code: EnrollmentService.java - `markPaid`; FileRepository.java - `writeRecords`

Group questions

Who did what?

I use member headers to explain the agreed module split, with integration under IT26101135. I do not treat a header as proof that only one person edited a file.

Show in code: AuthController.java - `class declaration`; InstructorSorter.java - `sort`; BookingRequestQueue.java - `enqueue`; DemoDataSeedRunner.java - `run`; EnrollmentService.java - `markPaid`; AdminReportController.java - `export`

How did we use Git and pull requests?

NEEDS CHECK: I can show local merge subjects mentioning PR 36 and PR 32, but cannot confirm every review or author action from code. I would ask our lead to verify the exact team workflow before describing it as our history.

Show in code: GIT_BRANCH_PLAN.md - `planned workflow`

How did we test together?

I can show the recorded final build, JUnit and MockMvc tests, and isolated browser evidence. I need the team to confirm who performed each rehearsal before claiming a personal contribution.

Show in code: AdminReportServiceTest.java - `class declaration`; AuthWebIntegrationTest.java - `class declaration`

How did we handle merge conflicts?

NEEDS CHECK: I can see a local merge subject about resolving instructor contracts. I cannot infer who resolved each conflict or the discussions, so I would verify that with the team.

Show in code: MERGE_ANALYSIS.md - `historical analysis`

What was the hardest bug?

NEEDS CHECK: I cannot know which bug the team found hardest without their account. I can explain the tested future-completion and payment-concurrency risks and show the current guards.

Show in code: Booking.java - `complete`; EnrollmentService.java - `requireAwaiting`; EnrollmentPaymentServiceTest.java - `class declaration`

Questions I must not bluff

Payments

I confirm payments manually after the school checks receipt information. I do not claim a gateway or bank reconciliation.

Show in code: EnrollmentService.java - `markPaid`

Email and receipts

I do not send email or upload receipts. I display contact instructions only when configured.

Show in code: EnrollmentController.java - `payment`

Password recovery

I use an in-memory reset code shown locally. I do not claim verified email ownership, reset-email delivery or production-ready recovery.

Show in code: PasswordController.java - `requestReset`; PasswordManagementService.java - `createResetToken`

Storage and locking

I use files and locks inside one JVM. I do not claim multi-file transactions, cross-process locking or database rollback.

Show in code: FileRepository.java - `writeRecords`; AccountWriteCoordinator.java - `execute`

Roles

I explain four user experiences, but only three stored roles. I do not invent a Visitor enum.

Show in code: Role.java - `class declaration`

Reports

I group revenue by enrollment month because payment dates are not stored. I do not claim an atomic multi-store report snapshot.

Show in code: AdminReportService.java - `business`

Seed and claims

I use fictional identities and syntax-only phone/NIC patterns. I do not claim real accreditation, licences, pass rates or real student feedback.

Show in code: DemoDataSeedRunner.java - `seedAccounts`

Team history and coverage

I mark unverified PR reviews, conflicts and personal contributions NEEDS CHECK. I do not say passing tests prove every browser state or complete WCAG compliance.

Show in code: GIT_BRANCH_PLAN.md - `planned workflow`